

The HLD Interview Framework

Phase 4 · Interview-Ready · Estimated time: 1.5 hours

1. Concept From First Principles

Phases 1 through 3 taught you individual concepts — load balancers, sharding, consensus, event sourcing — largely in isolation, one chapter at a time. A real HLD interview never asks you to explain a concept in isolation. It hands you a vague, open-ended prompt ("design WhatsApp," "design an e-commerce checkout") and gives you 45–60 minutes to produce a coherent architecture, live, out loud, while someone watches and probes your reasoning. The skill that separates a pass from a fail is rarely knowledge of any single concept — it's almost always the ability to run a **repeatable process** under time pressure so you never freeze, never wander, and never run out of time before reaching the parts that actually differentiate a strong candidate.

This chapter gives you that process: a six-step framework used, in slightly different words, by nearly every strong system design interviewer and candidate. Chapters 3 onward apply this exact framework, step by step, to fifteen real mock problems — so treat this chapter as the skeleton that every remaining chapter in this Phase hangs on.

Backend engineer analogy

Think of this framework the way you think of a Laravel Artisan command's lifecycle, or a well-structured PR description: boot → validate input → run core logic → handle edge cases → return response. You don't improvise that structure fresh for every controller you write — you follow the same shape every time, because the shape itself is what keeps you from missing a step under deadline pressure. The HLD framework is the same discipline applied to a 45-minute conversation instead of a codebase.

Step 1 — Clarify requirements (5–7 minutes)

The single most common way strong engineers fail this interview is skipping this step and diving straight into architecture. A prompt like "design WhatsApp" is deliberately underspecified — the interviewer wants to see whether you ask questions before you design, the same way a good engineer clarifies a vague Jira ticket before writing code instead of guessing and rebuilding later. Split your questions into two buckets. **Functional requirements:** what must the system do — 1:1 chat only, or also group chat? Media sharing? Read receipts? **Non-functional requirements:** what quality attributes matter most — is this optimized for low latency, strong consistency, massive read scale, or write-heavy ingestion? Explicitly state which non-functional requirements you're prioritizing and which you're consciously deprioritizing (Phase 1, Chapter 2's CAP-style trade-off framing) — this single sentence signals seniority more than almost anything else you'll say in the interview.

Step 2 — Estimate scale (3–5 minutes)

Back-of-envelope math converts a vague prompt into concrete numbers that drive every later decision. Estimate daily active users, requests per second (average and peak), read:write ratio, and storage

growth per day/year. You don't need precision — you need order of magnitude, stated out loud, with your assumptions visible. "Let's assume 50 million DAU, each sending 20 messages a day, so roughly 1 billion messages/day, which is about 11,500 writes/second average, and I'll assume peak is 5x average for a launch event or a cricket-final-night spike, so design for roughly 58,000 writes/second at peak." That one sentence does more work than any diagram: it tells you (and the interviewer) whether you're building a single-server toy or a system that needs Phase 1's sharding and Phase 2's message queues from day one.

Step 3 — Define the API / interface (3–5 minutes)

Sketch the core API endpoints or interfaces the system exposes — just enough to pin down what data flows in and out, without over-engineering REST conventions live on a whiteboard. For a chat system: POST /messages {senderId, receiverId, content}, GET /messages?conversationId=X&cursor=Y. This step is small but valuable: it forces you to nail down the exact shape of the data before you design storage for it, the same discipline as writing a route file and request validation before writing a Laravel controller's business logic.

Step 4 — High-level design (15–18 minutes)

This is the core of the interview: draw the major components (client, load balancer, API gateway, application services, databases, caches, message queues, CDN) and how data flows between them for the primary use cases identified in Step 1. Narrate as you draw — silent drawing loses the interviewer, since the interview is testing your reasoning, not your final diagram. Start simple (a monolith-shaped flow is a fine starting sketch) and evolve it, out loud, as scale demands — this narrated evolution is more convincing than jumping straight to the fully-scaled final architecture, because it proves you understand *why* each additional piece (cache, queue, shard) earns its complexity rather than just reciting a memorized diagram.

Step 5 — Deep dive (10–15 minutes)

The interviewer will steer you into one or two components for genuine depth — this is where Phases 1–3's specific concepts get spent. "How exactly does your database schema handle this?" "Walk me through what happens if this service crashes mid-write." "How would you handle a celebrity account with 50 million followers?" Answer with the specific pattern (idempotency keys, event sourcing, hybrid fan-out) and, critically, the trade-off you accepted to use it — not just the pattern's name. Naming a pattern without its trade-off reads as memorized; naming the trade-off reads as understood.

Step 6 — Trade-offs, bottlenecks, and wrap-up (3–5 minutes)

Proactively name what you'd do differently with more time, what the current design's weakest point is, and how you'd monitor it in production (Phase 3, Chapter 13's observability). Interviewers consistently report that candidates who volunteer their own design's weaknesses read as more senior than candidates who wait to be asked — it demonstrates the self-critical judgment that's much harder to fake than surface-level knowledge.

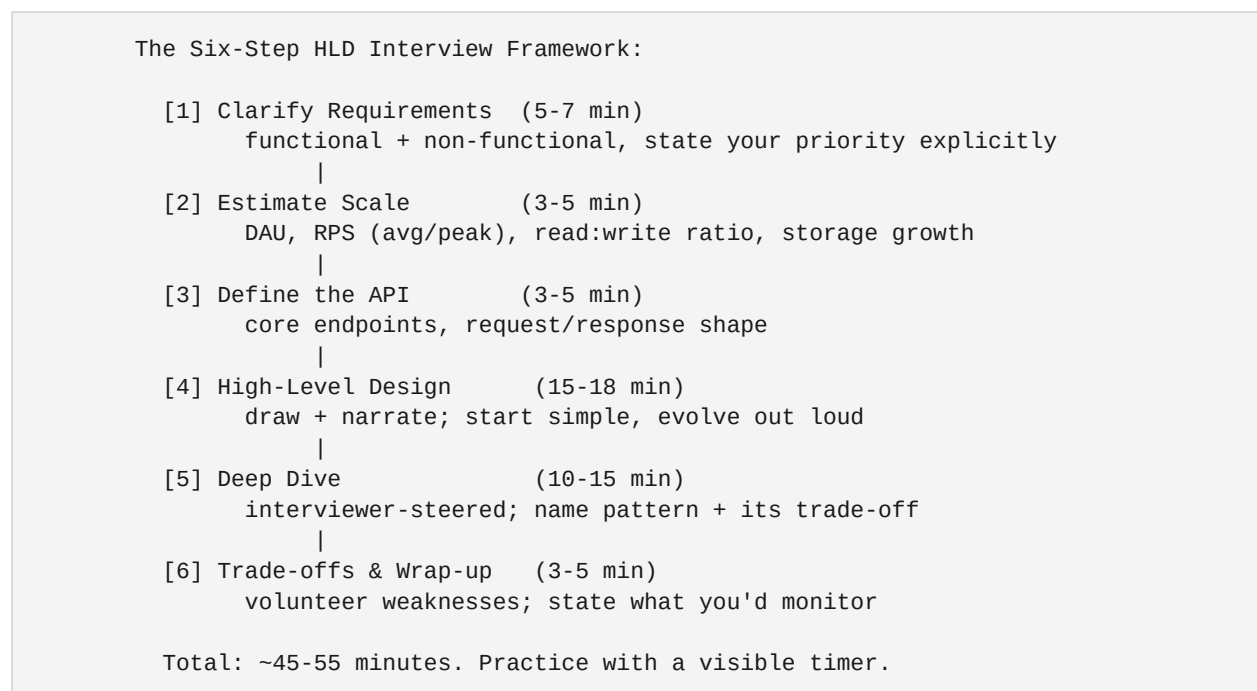
2. Real-World Example / Case Study

Engineers who've conducted or passed system design interviews at Flipkart, Swiggy, and Razorpay describe a consistent pattern: candidates who spend the first 5 minutes clarifying requirements and

estimating scale almost always produce a more coherent 45-minute answer than candidates who start drawing boxes immediately, even when the second group technically knows more individual concepts. The clarification step isn't a formality — it's the step that prevents you from designing the wrong system entirely, which is unrecoverable once 30 minutes have passed.

A frequently cited real failure mode at Careem- and Noon-tier interviews: a candidate asked to "design a ride-hailing dispatch system" spends 20 minutes building a beautifully detailed chat-style messaging architecture between rider and driver, because they never confirmed whether real-time chat was even a required feature. The interviewer had assumed dispatch and matching were the focus. That 20 minutes is gone, and the candidate never reaches the actual differentiating deep dive on geo-indexing and matching algorithms. This is precisely the failure Step 1 exists to prevent.

3. Diagram



4. Trade-offs Table

Step	Risk if skipped	Risk if over-invested
Clarify requirements	Design the wrong system entirely	Burns time before any design is shown
Estimate scale	Design decisions look unjustified/arbitrary	Over-precision reads as stalling
Define the API	Storage design decided before data shape is clear	Whiteboard REST bikeshedding
High-level design	Nothing concrete to deep-dive into later	No time left for the deep dive
Deep dive	Interview stays shallow, hard to differentiate	Tunnel-visions on one component, ignores rest

Step	Risk if skipped	Risk if over-invested
Trade-offs/wrap-up	Reads as unaware of your own design's limits	Rare risk — usually too little time is spent here, not too much

5. Common Interview Questions

Q1: "Design WhatsApp." Where do you start, in your first 30 seconds?

Model approach: Don't start with architecture. Say out loud that you're going to spend a few minutes on requirements first, then ask 2–3 sharp clarifying questions (1:1 only or also groups? Read receipts required? What scale?) before touching the whiteboard.

Q2: "You're running low on time. What do you cut?"

Model approach: Never cut Steps 1–3 — they're fast and everything depends on them. If time is genuinely short, compress Step 4 to the essential path only and go straight to one focused deep dive (Step 5) on the component you already flagged as most interesting, rather than spreading thin across many shallow components.

Q3: "How do you decide which component the interviewer wants you to deep-dive on?"

Model approach: Listen for the interviewer's own follow-up questions during Step 4 — they usually signal it directly ("how would that handle X?"). If they don't, proactively suggest 1–2 candidates yourself ("I'd like to go deeper on either the matching algorithm or the location-update pipeline — which is more useful to you?"), which itself signals strong judgment about where the real complexity lives.

6. Practice Exercises

- 1 Set a timer for 5 minutes and write out every clarifying question you'd ask for "design Instagram," split into functional vs. non-functional buckets.
- 2 Do a full back-of-envelope estimation (DAU, RPS average/peak, storage/year) for a hypothetical Indian ride-hailing app with 20 million monthly active users.
- 3 Pick any Phase 2 or Phase 3 chapter topic and, in under 2 minutes out loud, explain the pattern AND the trade-off you'd accept to use it — practicing Step 5's core skill in isolation.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Complete System Design Roadmap for 90 Days FAANG Interviews"	Frames the overall interview process this framework fits into.
English	"System Design Interview Verticals — How Big Tech Engineers Think About Architecture"	Reinforces the requirements-first, narrate-as-you-draw discipline this chapter teaches.
Hindi	"Follow this for every system design interview" — Chai aur Code	Hindi-language walkthrough of a repeatable step-by-step interview framework, directly matching this chapter's structure.

Language	Video & Channel	Why it helps
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Full bilingual course; useful to revisit the framework applied across many topics.

8. Key Takeaways

- A repeatable six-step framework — clarify, estimate, API, high-level design, deep dive, trade-offs — beats improvising fresh structure under time pressure.
- Skipping requirements clarification is the single most common cause of a strong engineer failing this interview.
- Back-of-envelope estimates aren't about precision; they're about giving your later design decisions visible justification.
- Narrate while you draw. A silent whiteboard tests your final diagram; an interview tests your live reasoning.
- In the deep dive, name the trade-off you accepted, not just the pattern's name — that's what reads as understood rather than memorized.
- Volunteering your own design's weaknesses in the wrap-up reads as more senior than waiting to be asked.
- Every remaining chapter in this Phase applies this exact six-step framework to a real, fully-worked mock problem.